network

MDPI

Article

Evaluating AES-128 Segment Encryption in Live HTTP Streaming Under Content Tampering and Packet Loss

Bzav Shorsh Sabir [1,*] and Aree Ali Mohammed [2]

[1] Department of Information Technology, Technical College of Informatics, Sulaimani Polytechnic University, Sulaymaniyah 46001, Iraq
[2] Computer Science Department, College of Science, University of Sulaimani, Sulaymaniyah 46001, Iraq; aree.ali@univsul.edu.iq
* Correspondence: bzav.shorsh.tci@spu.edu.iq

Abstract

One of the main sources of entertainment is live video streaming platforms, which allow viewers to watch video streams in real time. However, because of the increasing demand for high quality content, the vulnerability of streaming systems against cyberattacks highlights how crucial it is to implement strong security mechanisms without sacrificing performance. Therefore, the safeguard of video streams against cyberthreats such as content tampering and interception is a top priority while still maintaining robustness against network fluctuations. Two distinct scenarios are proposed to test AES-128 encryption in securing HTTP live streaming segments against content tampering and resilience to packet loss. Results show that AES-128 encryption provides confidentiality and successfully prevents meaningful manipulation of the video content, confirming its reliability as segment encryption does not significantly alter packet loss-induced playback behavior compared to unencrypted streaming under the tested conditions, Performance analysis shows that AES-128 has no significant difference in data loss for up to 4% of network packet loss compared to unencrypted segments.

Keywords: content distribution network; adaptive bitrate streaming; cybersecurity; quality of experience; man-in-the-middle attack

Check for updates

Academic Editor: Panayiotis Kotzanikolaou

Received: 18 November 2025
Revised: 22 December 2025
Accepted: 30 December 2025
Published: 31 December 2025

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The growth of mobile devices and the ubiquitous availability of high-speed internet has led to a continuous increase in the use of video streaming [1]. The rise in multimedia distribution platforms such as Twitch and YouTube has resulted in a significant surge in new social networking models. Moreover, contemporary user gadgets including diverse computing capabilities and display resolutions effectively ensure customer pleasure regarding on-demand video quality adaptation. Personal live streaming content can be generated by any individual. Home TVs provide live entertainment through a variety of streaming apps created by third parties and content providers [2].

From a technological standpoint, Live Video Streaming (LVS) refers to a service that distributes video content that simultaneously captures and broadcasts video materials to viewers in real time. Unlike Video on Demand (VoD), live streaming offers real-time interaction and engagement, which appeals to modern audiences seeking immediacy and connection. To provide convenient service experiences, LVS systems are generally deployed on internet infrastructure utilizing web transfer protocols to simultaneously disseminate

video packets across different pathways. In a contemporary LVS system, diverse user demands and preferences are accommodated via adaptive bitrate streaming services, which allow networks to dynamically modify video quality based on environmental circumstances and resource availability [2,3].

In video streaming systems, the CIA triad highlights key security areas: confidentiality, which is about protecting data from unauthorized users, integrity, which is making sure the data is not manipulated, and availability, which is ensuring the data is available when requested [4]. Many encryption techniques have been used as a result of the necessity to shield sensitive data from unauthorized users. Nonetheless, encrypting the video content that needs to be sent is crucial [5]. A key method for preventing unauthorized access to digital data and guaranteeing its integrity and secrecy is encryption [6,7]. In 2001, NIST developed the Advanced Encryption Standard (AES), a symmetric block cipher, to replace the antiquated Data Encryption Standard (DES). AES supports key sizes of 128, 192, and 256 bits, operating on 128-bit blocks, and is widely used in secure communications and storage. AES-128 is the most common variant, offering a balance between security and efficiency [8].

Malicious modification of a digital video is known as content tampering, or video forgery. As seen in Figure 1, there are two key areas where this can be accomplished:

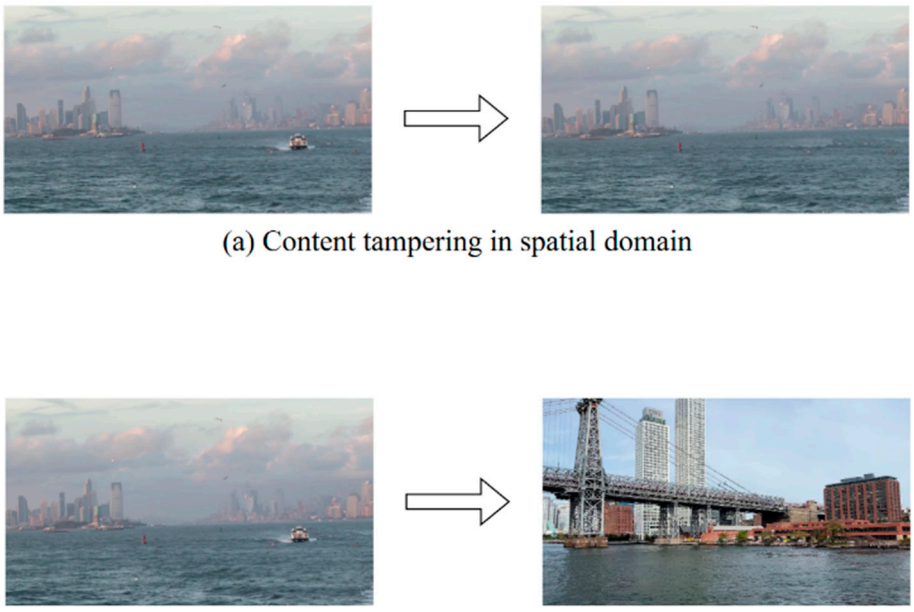

Figure 1. Modification in spatial and temporal domain.

Spatial domain: changing the content of frames (e.g., removing an object within a frame).

Temporal domain: modifying the frame sequence across time (e.g., inserting, deleting, duplicating, or reordering frames).

The goal of content tampering aims to manipulate the video to change its meaning for malicious reasons, such as spreading misinformation [9].

Although encryption improves integrity and confidentiality, it can also result in computational overhead that degrades network performance and, in turn, Quality of Experience (QoE). The entire level of user satisfaction with an application or service is measured by QoE. As opposed to Quality of Service (QoS), which places more emphasis on technical aspects that can be measured, such as bandwidth, latency, jitter, and error rates [10]. Therefore, video streaming platforms must strike a balance: maintaining strong security to preserve confidentiality, while also ensuring sufficient QoS to deliver a high QoE. This

trade-off becomes especially critical in live streaming scenarios, where real-time interaction leaves little tolerance for delays, buffering, or retransmissions.

The conventional approach to streaming involves the use of protocols like RTMP that transmit content using User Datagram Protocol (UDP). But when it comes to distributing information to environments that encompass many platforms, companies confront considerable hurdles [11,12]. The newer technique of streaming is by using HTTP Adaptive Streaming (HAS) protocols, HAS simplifies the delivery of content by using Hypertext Transfer Protocol (HTTP), it takes advantage of standard web servers or caches found in networks of Internet Service Providers (ISPs) and Content Distribution Network (CDN) [11]. HAS became the standard of video streaming due to its adaptability to varying network conditions [13]. The server is not responsible for the client state, therefore the client can download the segments from different servers, and it eliminates the need for a persistent connection between the client and the server [11]. Given the network bandwidth, this adaptive approach guarantees that the viewers receive the most optimal quality video based on their bandwidth [7,14]. HAS employs HTTP as the application protocol and Transmission Control Protocol (TCP) as the transport-layer protocol, with clients retrieving data from a conventional HTTP server that hosts the media content. HAS systems adaptively select the appropriate video bitrate to accommodate fluctuating network circumstances [11,15]. Today, HAS accounts for the majority of internet video traffic [11]. It has reached mainstream due to commercial solutions such as HTTP Live Streaming (HLS), Microsoft Smooth Streaming (MSS), Adobe's HTTP Dynamic Streaming (HDS), and Dynamic Adaptive Streaming over HTTP (DASH) [16].

Apple developed HLS [17] in 2009, which is a video streaming protocol to reliably send continuous video over the internet. It is one of the HAS protocols which uses TCP as the transport layer protocol and HTTP as the application [11]. With this pull-based streaming technique, the video is divided into segments and encoded at various bitrates. Additionally, the server creates a manifest file (referred to as playlist) that includes the video, and subtitle metadata as well as the places where they can be retrieved. In order to optimize QoE, the client chooses to pull the relevant bitrate chunks based on available bandwidth [11,12,18].

Although prior studies have improved adaptive streaming and security independently, little research has explored the trade-offs between segment-level encryption and Quality of Experience in large-scale CDN environments under real-world attack scenarios. This work builds upon the experimental testbed introduced [19], which focused on CDN optimization and packet loss effects in live HLS without considering security mechanisms. While the underlying testbed architecture is reused for consistency and comparability, the present study introduces segment-level AES-128 encryption, a man-in-the-middle threat model, and controlled content tampering experiments that were not part of the prior work. The research contributions include (1) simulating Man-in-the-Middle (MitM) attacks to evaluate the effect of AES-128 encryption on confidentiality, integrity, and availability, and (2) analyzing the trade-offs between security and QoE under network packet loss condition using metrics like Peak Signal-to-Noise Ratio (PSNR), data loss ratio, and subjective playback evaluation.

This paper's structure is set up as follows: Section 2 reviews relevant research on security in video streaming, while Section 3 introduces the suggested methodology. The experimental data are thoroughly analyzed in Section 4, and the results are discussed and contrasted with previous studies in Section 5. Section 6 brings the study to a close and suggests possible avenues for further investigation.

2. Related Works

In recent years, video streaming platforms have continued to grow across diverse devices and networks. However, they face various security challenges, such as unauthorized access, content privacy and data interception. To increase security without compromising streaming systems' performance or scalability, researchers and developers have been looking into advanced cryptographic techniques, selective encryption strategies, and hardware-assisted architectures.

Yun et al. [20] introduced Jumble Lightweight Video Encryption Algorithm (JLVEA), which is a lightweight video encryption algorithm designed specifically for low-power IoT devices such as security cameras. JLVEA performs encryption prior to compression to preserve frame readability even in the event of packet loss, and it separates and permutes individual RGB color channels to prevent attackers from deducing content from color composition. JLVEA outperforms previous methods, achieving higher MSE values, faster encryption speeds than other pre-compression approaches, and lower memory usage than algorithms requiring pre-generated permutation lists. Building upon the idea of efficient and secure video encryption. Alawi et al. [21] presented a selective video encryption technique that employs the ChaCha stream cipher, designed to enhance the speed and security of video encryption for substantial datasets. The main contribution is adding the Features from Accelerated Segment Test (FAST) feature detection operator to find and encrypt only the most important key spots in video frames, instead of encrypting every pixel. The authors created three encryption modules: full-frame encryption, partial encryption with FAST key points, and an improved partial encryption that includes points next to FAST-detected key points. The experimental evaluation showed that the selective ChaCha-based methods, especially with the FAST enhancement, saved a lot of time while still being very secure.

The traditional method of AES encryption that use CPU suffers from memory bandwidth constraints, to solve this issue, Liu et al. [22] presented AESPIM, a Processing-In-Memory (PIM) architecture engineered to enhance the speed of AES encryption for online video streaming. By shifting memory-intensive AES computations to the memory side via Hybrid Memory Cube (HMC) technology, it significantly reduces the amount of data sent between the CPU and memory. The introduction of a QoS-aware scheduler that optimizes workload balancing among memory vaults adds an additional 6% gain, the investigation of user-level (inter-user) parallelism adds an additional 34% improvement, and a basic design that boosts performance by up to 42% over CPU-based implementations are among the main contributions. Guo et al. [23] systematically investigated adversarial attack and defense mechanisms for Deep Learning-based Soft Sensors (DLSSs) by proposing a black-box, Knowledge-Guided Adversarial Attack (KGAA) framework. By incorporating domain-specific mechanism knowledge into the optimization objective, the proposed approach addresses the ill-posed nature of adversarial attacks on regression-based models and enables the generation of highly imperceptible and stable adversarial perturbations. In addition, a corresponding adversarial training strategy was introduced to proactively enhance the robustness and reliability of DLSSs under attack.

Usmani et al. [24] introduced a Digital Rights Management (DRM) framework that shifts the security focus from the host device to the video stream. It employs Ciphertext-Policy Attribute-Based Encryption (CP-ABE) to protect video material based on user attributes such as location, role, or subscription level. AP-ABE can be used for video streaming without requiring reliable hardware or software while striking a balance between scalability, policy enforcement and robust access control. The system's prototype shows that it significantly increases security and flexibility in video streaming while having little effect on performance. Recently, Sabir et al. [19] explored improving QoE in live

video streaming under packet loss conditions by tuning HLS parameters and applying load balancing in a simulated environment with CDN included. The study evaluated how segment length, list length, and Group of Pictures (GOP) size affect resilience to packet loss, alongside comparing round robin and ring hash algorithms. The results show that longer segment and list lengths reduce data loss but increase stream delay, while GOP size has a smaller yet context-dependent impact. Ring hash consistently outperformed round robin, reducing data loss to below 1.4% under 5% packet loss. Table 1 summarizes the mentioned related works.

Table 1. Summary of related works.

Reference	Contribution	Environment	Result
[20]	Introduced JLVEA	Real World	JLVEA achieves higher MSE values, faster encryption speeds than other pre-compression approaches, and lower memory usage than algorithms requiring pre-generated permutation lists.
[21]	Proposes a selective video encryption method	N/A	The selective approach achieved significantly faster encryption times.
[22]	Presented AESPIM	Simulation	Up to 42% performance improvement over CPU-based AES, plus 34% from user-level parallelism, and 6% from QoS scheduling.
[24]	Proposes ABEVS	Real World	Reduced cache CPU load by up to 50% compared to HTTPS, while maintaining similar video quality (VMAF) and cache hit rates.
[19]	Tuning HLS parameters and load balancing	Simulation	Ring hash load balancer and optimal HLS parameters kept data loss below 1.4% even under 5% packet loss.

Prior studies did not combine security and CDN optimization in one testbed, nor did they simulate both encryption and packet loss scenarios.

3. Materials and Methods

The testbed aims to establish a realistic environment for the deployment, testing, and evaluation of a video streaming platform that is secure and optimized through a CDN. It is the same testbed used in [19] but configured for segments to be encrypted. The architecture comprises a primary server for video streaming, cache server for enhancing delivery of content, a load balancer to distribute incoming requests, an attacker node for simulating cyberthreats, and client machines for video download. The virtual machines constitute the primary components of the system. Unlike VoD, the nature of live streaming introduces the challenge of not tolerating significant delays, buffering or retransmission errors. Any disruption can degrade the viewing experience or cause users to abandon the stream altogether. Additionally, because live content is transmitted as it is being recorded and encoded, there is limited opportunity for pre-processing, redundancy, or fallback strategies [10]. The number and existence of the components vary depending on the scenarios.

3.1. Video Characteristics

The source video has a resolution of 1920 × 1080, a duration of 4 min and 32 s, and a frame rate of 24 fps. It is encoded in real time with FFmpeg (FFmpeg team, Paris, France) into three different bitrates using the H.265 codec and the ultra-fast preset: a high-quality version with a bitrate of 4.8 Mbps, resulting in approximately 160 MB total size, a medium-

quality version at 3 Mbps with around 100 MB size, and a low-quality version at 1 Mbps producing a file of about 35 MB. The playlist length is 10 or 5 segments and segment length is 1.5 s with a GOP size of 12 frames. The selected parameter configuration results in a low end-to-end delay, approximately 4.5 to 7.5 s for the playlist length of 5 segments and 4.5 to 15 s for playlist length of 10 segments while still providing sufficient buffering to tolerate moderate packet loss.

3.2. Key Management

AES-128 encryption is applied in Cipher Block Chaining (CBC) mode following the HLS specification. Initialization vectors (IVs) are automatically generated by the HLS packager and explicitly included in the EXT-X-KEY directive. IV generation and rotation are handled by the packaging tool and are not manually configured in this study. The operating system's cryptographically secure random number generator is used at the origin server to create a single 128-bit AES key. Throughout the experiments, the same encryption key is used for every streaming session. This design decision was made in order to preserve consistency between test cases and to separate the effect of AES-128 segment encryption on packet loss behavior and content tampering resistance without adding variability from key rotation techniques. The EXT-X-KEY directive is used in the HLS playlist to refer to the encryption key. To ensure confidentiality during key delivery, clients request the decryption key via an HTTPS connection. Key rotation, periodic re-keying, or per-session key creation are not implemented in order to prevent unnecessary computational and signaling overhead that can affect QoE evaluations. It is important to note that AES-128 in CBC mode provides confidentiality only and does not offer cryptographic integrity or authenticity guarantees.

3.3. Network Design

3.3.1. Origin Server

The origin server is the primary source of video material that is available on the network. It is set up as a virtual machine that has 8 logical CPU cores and 4 GB of RAM allocated to, that runs on Debian 12 and is in charge of processing and delivering media content. To encode the video stream and segment the video into multiple HLS segments in real time, the origin server uses FFmpeg. The origin server is also very vital to assure safety of the contents by encrypting the segment using AES-128. The Nginx (F5, Inc., Seattle, WA, USA) web server, which has been set up to enable HLS, then makes the contents accessible to clients. This configuration is like what would occur in the real world, where the origin server serves as the central pillar of the streaming infrastructure and generates and transmits material to cache servers or directly to clients when requested. All HLS communication—including playlist delivery, media segment transfer, and encryption key retrieval is protected using TLS 1.3.

3.3.2. Cache Servers

To improve performance, and distribute the network load, 50 docker servers running Nginx are created that act as cache servers that are hosted within a Graphical Network Simulator 3 (GNS3) VM (GNS3 Technologies Inc, Austin, TX, USA), which is a network experimentation framework that facilitates multivendor models and device emulation that provides network emulation capabilities [25]. 6 logical CPU cores and 4 GB of RAM were allocated to the GNS3 VM. In the event that a client makes the initial request for anything, every cache server is configured to retrieve the requested item from the origin server, store it on its own server, and then serve it to other clients. The application of this caching technology reduces the strain on the origin server.

3.3.3. Load Balancer

Using the ring hash method, the load balancer had 2 logical CPU cores and 4 GB of RAM. It is a Debian 12 virtual machine running EnvoyProxy, distributes client requests to one of the cache servers. Consistent hashing is implemented by ring hash, in which each request is routed to a host by hashing a request property and finding the nearest matching host clockwise around the ring [26]. Ring hash causes the least amount of data loss when servers are added or deleted, improving QoE.

3.3.4. Attacker

The attacker is modeled as a MitM with trusted interception capability. Specifically, the attacker machine running MitMProxy is configured as a trusted root certificate authority on the client, allowing it to terminate and re-establish TLS connections between the client and the origin server. This controlled configuration enables interception and modification of HTTP payloads while preserving the use of HTTPS, reflecting a threat model in which the attacker has compromised trust on the client side rather than bypassing transport-layer encryption. The attacker is assumed to have full MitM capability on the delivery path, including the ability to intercept and modify TLS-protected traffic via a compromised trust anchor, but is assumed not to obtain encryption keys or compromise the key server. 4 logical CPU cores and 4 GB of RAM was allocated to the attacker machine, which also runs on Debian 12. In order to modify video segments as it is being transmitted, MitMProxy collaborates with a custom Python 3.11.2 script.

3.3.5. Clients

The client machine also runs on Debian 12 that has 8 logical CPU cores and 8 GB of RAM allocated to, it is used to mimic clients watching live streaming contents. FFmpeg can be used by one client to download and play HLS stream from either the cache servers or the origin server. Java-based automation is added with FFmpeg to make it several users at once and to improve testing scalability

3.4. Simulation Scenarios

Two distinct scenarios have been designed to analyze the robustness of the testbed against content tampering and packet loss. Table 2 shows the differences between scenario 1 and 2.

Table 2. Details of scenario 1 and 2.

Test Parameters	Scenario 1	Scenario 2
Number of clients	1 client	60 clients
Existence of attacker	1 attacker	No attacker
Number of cache servers	0 cache servers	50 cache servers
Usage of load balancer	Not used	Used
Playlist length	10 segments	5 and 10 segments
Evaluation metrics	Subjective evaluation and PSNR (graph)	Data loss
Total tested cases	3 cases	12 cases

3.4.1. Scenario 1: One Client and One Attacker

In the first scenario, the client requests the origin server to fetch the contents, but an attacker is placed in between the client and the origin server as seen in Figure 2. It is manually set up that requests are forwarded to the attacker to simulate a MitM attack, then the attacker tampers the content. Three cases are tested while the streamed video is sent:

- The attacker performs MitM attack by modifying one out of every 10 segments (TLS and AES-128 is disabled).

- The attacker performs MitM attack by modifying one out of every 10 segments while the segments are encrypted using AES-128 from the origin server.
- The attacker performs MitM attack by modifying the master playlist.

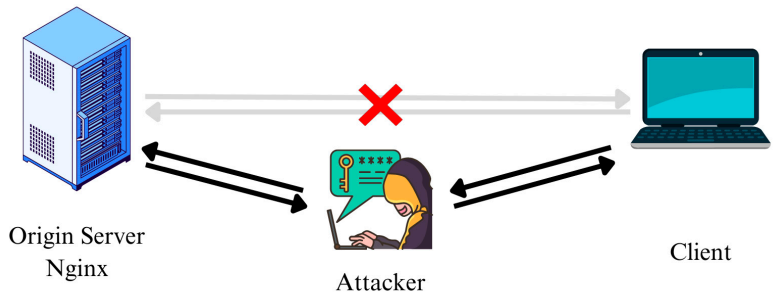

Figure 2. Network diagram of scenario 1, the client's requests are forwarded to the attacker, who intercepts and tampers with the content before relaying it to the origin server. Arrows indicate request–response communication paths, and the red cross denotes the absence of direct client–server communication.

Figure 3 shows the flowchart diagram of this scenario. The client requests the origin server to view the stream, the requests become intercepted by the attacker and the attacker sends the request to the origin server. While this happens, the origin server continuously encodes the input video into multiple representations and also encrypts the segments (if segment encryption is enabled), the origin server responds to the attacker with the requested content, then the attacker tries to modify the content. And lastly, the attacker responds to the client with the modified content. The client requests segments until there are no more segments left to request. The goal of the segment encryption is to achieve confidentiality by not permitting the attacker to see the segments.

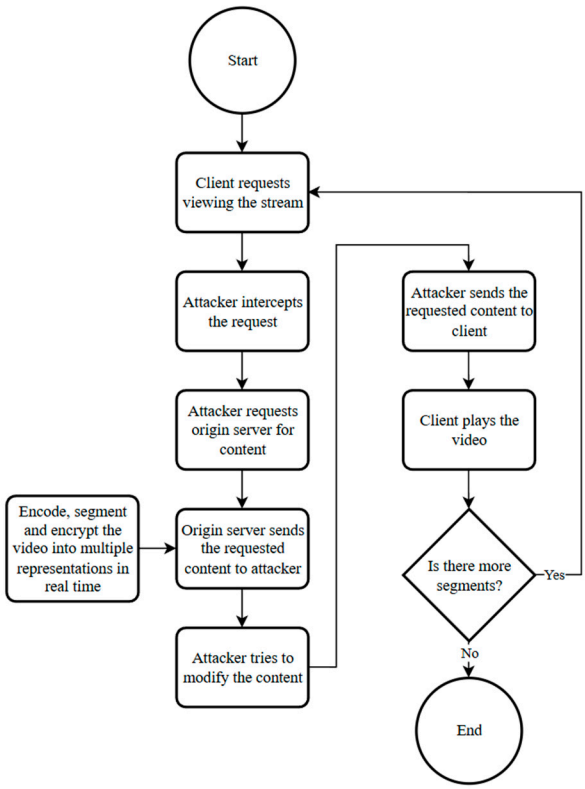

Figure 3. Flowchart diagram of scenario 1.

The choice of modifying one out of every ten segments represents a controlled and repeatable attack pattern rather than an attempt to model all possible real-world adversarial behaviors. The choice of this interval-based tampering technique allowed for the clear observation of both visual artifacts by introducing periodic integrity violations while maintaining stream continuity. Other attack patterns such as replay, segment reordering, or cross-representation swapping were not evaluated and are left for future work.

3.4.2. Scenario 2: 60 Clients and 50 Cache Servers

In the second scenario, 60 clients are simulated using java that simultaneously request to fetch the contents using FFmpeg from the load balancer. The load balancer distributes the requests using ring hash algorithm to the cache servers. Six cases are tested where packet loss is simulated from 1% to 6% to show the effect of packet loss on data loss while segments are encrypted. The network architecture for scenario 2 is depicted in Figure 4. The link between the cache server-load balancer and the main server-cache server is used to simulate packet loss.

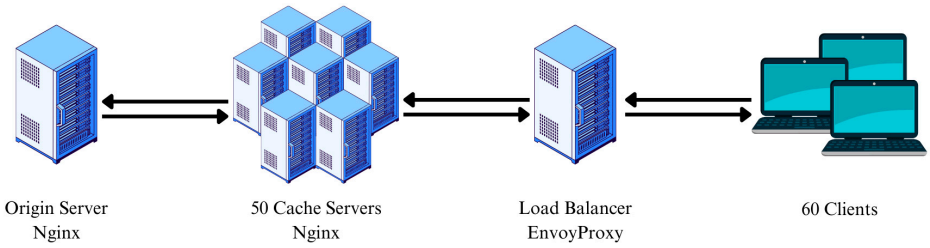

Figure 4. Network diagram of scenario 2.

The process diagram of scenario 2 is displayed in Figure 5. The client asks for the load balancer for the contents so that the stream can be pulled. The load balancer then asks a cache server, which verifies whether the content is available and responds with it if it does. If the material is not in the cache server, the cache server makes a request to the origin server, saves a copy, then transmits it to the load balancer, which responds to the client. This process is ongoing as the live video is continuously encoded, segmented, and encrypted into various representations by the origin server.

Packet loss rate is varied as the primary experimental parameter to evaluate its impact on playback behavior under encrypted and unencrypted streaming. Other network conditions are held constant across all experiments to isolate the effect of packet loss. Specifically, available bandwidth is provisioned to exceed the highest stream bitrate to avoid bandwidth-induced congestion, round-trip time is kept stable at a low-latency baseline representative of a local network environment, and default operating system TCP congestion control and buffer configurations are used on all nodes. No artificial delay, bandwidth throttling, or buffer manipulation is applied beyond the controlled packet loss simulation.

3.5. Performance Analysis Metrics

Assessing the perceived quality of video is a crucial part of assessing QoE in video streaming services [27]. One of the most common metrics used to assess image quality is PSNR, and the Mean Square Error (MSE) is its main constituent and the source of its formation [28]. For the first scenario, for evaluation of the tests conducted, PSNR is computed on a per-frame basis between the decoded video output and the original, untampered reference video, it is used as a comparative indicator of visible degradation rather than a comprehensive QoE metric. Subjective evaluation is also utilized to analyze and demonstrate the distinction in the situation in which the content is tampered while encryption is applied. To measure the effect of packet loss on data loss during transmis-

sion, the total size of the downloaded files is computed with the expected size for the second scenario.

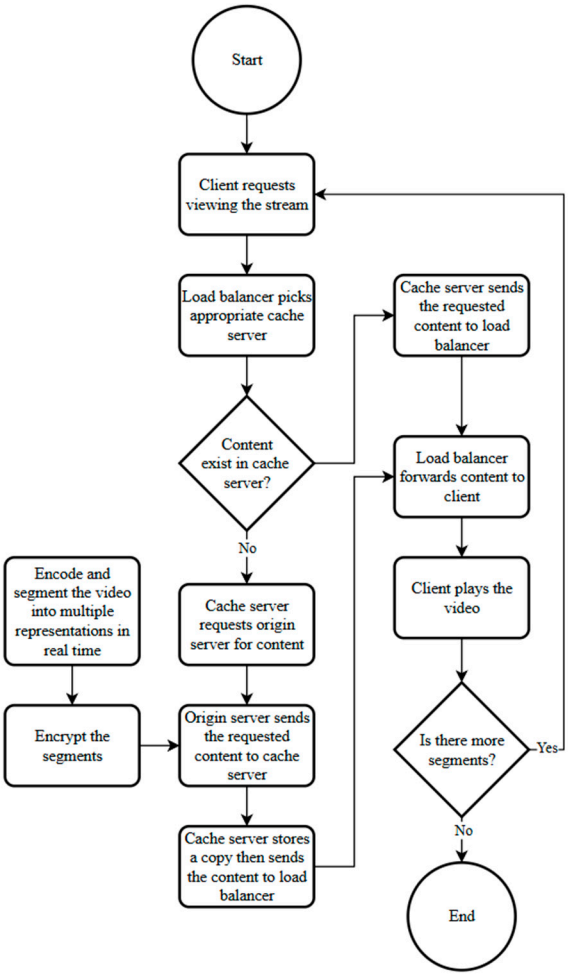

Figure 5. Flowchart diagram of scenario 2.

4. Results

4.1. Scenario 1

In the first scenario, a MitM attack is simulated to test how well content encryption works as a defense in a video streaming setting. An attacker intentionally sends all HTTPS requests and responses between the client and the origin server through their own machine while tampering with one out of every ten segments.

4.1.1. Case A: No Security Measures (Segments Not Encrypted)

In the first case, the origin server does not use any kind of encryption. This makes the whole communication stream visible in plain text, giving the attacker full access to and control over video segments as they are sent. Because of this, the attacker can successfully intercept and change one out of every ten segments and send the tampered version to the client. The client accepts these altered segments without checking them, which causes the stream to have modified segments. Visual inspection of the playback reveals evidence of tampering, as the affected video segments appear to be tampered rather than the expected content.

4.1.2. Case B: AES-128 Encrypted HLS Segments

In the second case, the origin server is set up to encrypt all HLS segments by AES-128. In this case, when the attacker pretends to be the client and asks for video segments, the origin server sends back encrypted content. To change any part, the attacker would have to perform the following actions:

1. Decrypt the segment.
2. Modify the segment.
3. Re-encode with the same codec and parameters.
4. Encrypt the segment using the same AES-128 key.
5. Forward the modified segment to the client.

But this process is almost impossible for the attacker because they do not have the decryption key. Because of this, trying to change the encrypted part without first decrypting it will corrupt it. The HLS player cannot decrypt or decode a corrupted segment that is sent to the client which leads to the client being unable to see the segment. This stops the attacker from changing the content and is a strong defense against MitM attacks. Figure 6 shows this behavior by showing how the PSNR drops at the client when the attacker tried to modify a segment while AES-128 encryption is on. Upon visual inspection of the playback, the tampered video segments did not display the intended content; instead, they manifested as black frames. This visual artifact highlights a discernible decline in the viewing experience that may not be well represented by numerical quality metrics, acting as a tangible indicator of content tampering. The observed decoding failure and appearance of black frames under ciphertext modification should not be interpreted as a formal cryptographic integrity guarantee. Rather, this behavior reflects the fact that unauthorized bit-level modifications to encrypted HLS segments result in undecodable content when decryption is attempted with the correct key. While this prevents an attacker from producing meaningful manipulated video content, it does not provide explicit authentication or tamper detection in the cryptographic sense.

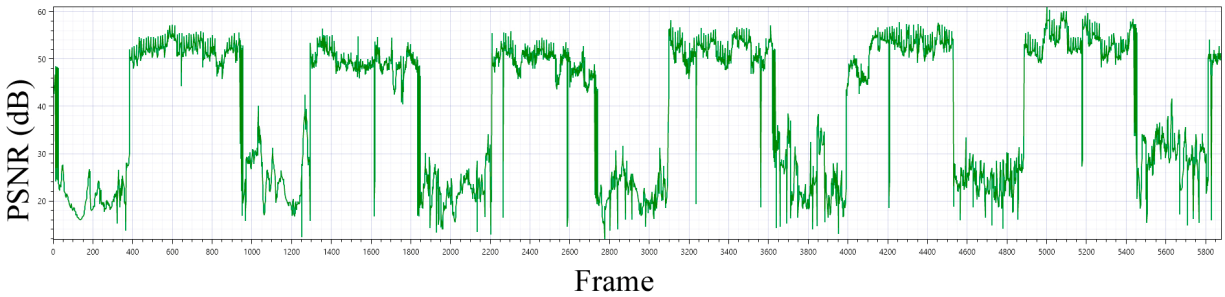

Figure 6. Scenario 1, frame-level PSNR over time under encrypted streaming with periodic segment tampering. Every tenth media segment is subjected to arbitrary ciphertext modification by the attacker, resulting in sharp PSNR drops and visible playback artifacts, while untampered segments maintain stable quality.

4.1.3. Case C: Master Playlist Tampering

In addition to segment-level ciphertext modification, tampering of the HLS master playlist was also evaluated. When the attacker modified the master playlist—such as altering variant stream references or representation ordering—the client accepted the modified playlist and adjusted stream selection accordingly. This demonstrates that, in the absence of playlist authentication, master playlist manipulation remains feasible even when media segments are encrypted. While this work does not provide a comprehensive evaluation of all possible playlist-level attacks, the observed behavior highlights that

segment encryption alone does not protect against control-plane manipulation at the manifest level.

4.2. Scenario 2

To more accurately show the impact of segment encryption with AES-128 on data transmission with packet loss, The simulation is repeated twice for each case and the average of each case is shown. Figure 7 shows the effect of packet loss on data transmission while the segments are encrypted. Up to 4% of packet loss, segment encryption does not seem to increase the impact of packet loss. However, when packet loss reaches 5% or 6%, the resulting data loss becomes highly significant. Playlist length of 10 segments compared to 5 segments, had a slight impact on decreasing data loss due to network packet loss.

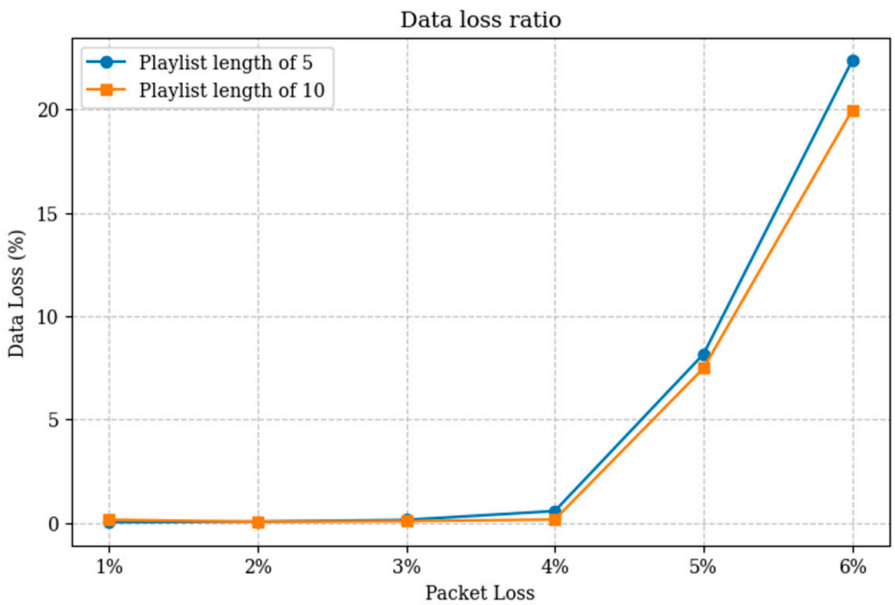

Figure 7. Scenario 2, data loss ratio as a function of packet loss rate for encrypted HLS with a segment length of 1.5 s and a GOP size of 12 frames. Results compare playlist lengths of 5 and 10 segments, showing that the longer playlist slightly reduces effective data loss at moderate to high packet loss rates by providing increased buffering tolerance.

5. Discussion

When AES-128 encryption is enabled for the segments with HLS, confidentiality is provided which prevents the attacker from seeing the segments, the attacker is also prevented from producing meaningful content by modifying the segments. However, the availability of the system is breached because when the attacker cannot modify the segments, it still can corrupt the segment, leading to the viewer seeing complete black frames for the duration of the segment length as seen in Figure 8. This result shows that the appearance of black frames under encrypted segment corruption highlights an integrity and availability issue rather than a confidentiality breach. Several mitigation strategies could be considered to reduce the impact of such attacks. At the content level, authenticated encryption or explicit integrity verification would allow tampered segments to be detected and rejected before decoding, enabling faster error recovery. At the streaming layer, shorter segment durations, increased playlist redundancy, or client-side retry mechanisms could limit the duration of visible playback disruption.

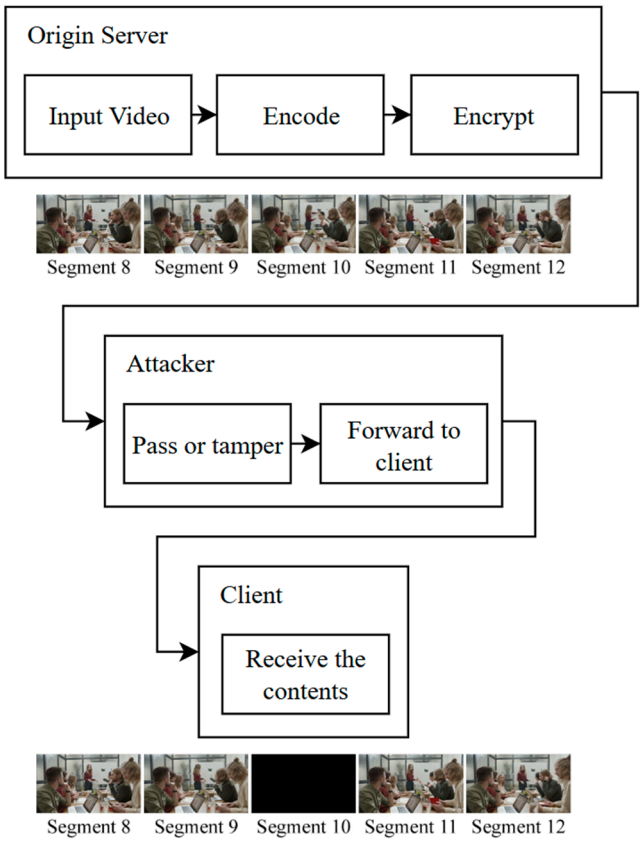

Figure 8. Scenario 1, Case B: Encrypted segment tampering under a MitM attack. Modification of encrypted selected segments results in a black frame at the client, while surrounding segments remain unaffected.

When multiple clients request a load balancer with ring hash algorithm that fetch the contents from cache servers, no noticeable difference is observed from data loss up to 4% of packet loss between segments encrypted with AES-128 and unencrypted segments. From 5% and 6% packet loss rate, the difference is significant enough to make a difference in QoE. Figure 9 shows the comparison of the data loss ratio when segments encrypted to when segments are not encrypted as tested in [19]. For segment length of 1.5 s with a GOP size of 12 frames while the load balancing algorithm is ring hash, a playlist length of 10 segments slightly outperforms playlist length of 5 segments in terms of data loss under network packet loss condition. However, it comes with a trade-off of 15 s of maximum delay compared to 7.5 s of delay when playlist length is 5 segments. Table A1 reports the numerical data loss values used to generate Figures 7 and 9.

The threat model considered in this study assumes that the adversary does not obtain the AES-128 segment decryption key. This reflects scenarios where key delivery is protected using TLS and the key server remains uncompromised. In practice, additional attack vectors may exist, including compromised key servers, misconfigured key URIs, or malicious or misbehaving CDN nodes with access to encryption keys. Such scenarios would allow an attacker to decrypt, modify, and re-encrypt segments, thereby bypassing the content-level tamper resistance observed in this work. These attack vectors are not addressed in the present study, as the focus is on evaluating the effects of ciphertext modification and packet loss under a protected key distribution model. Extending the analysis to key compromise scenarios represents an important direction for future work.

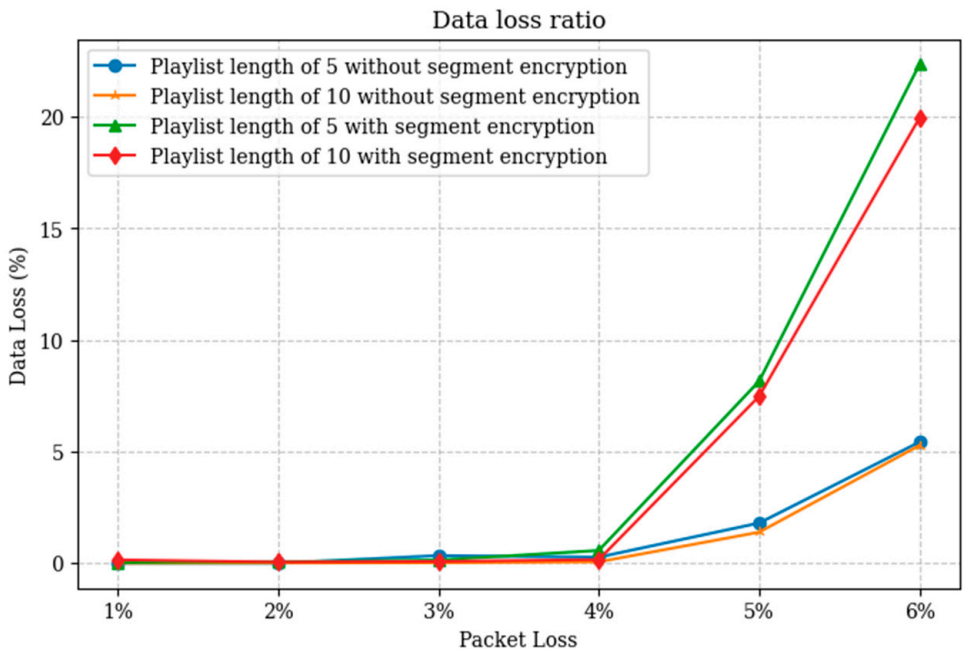

Figure 9. Scenario 2, Data loss ratio as a function of packet loss rate in, comparing encrypted and unencrypted HLS with a segment duration of 1.5 s and a GOP size of 12 frames. Results are shown for playlist lengths of 5 and 10 segments. Encrypted and unencrypted streams exhibit similar data loss trends at low to moderate packet loss rates, while encrypted streams experience higher data loss at increased packet loss.

6. Conclusions

Security in video streaming is a crucial aspect to prevent content tampering and preserve user privacy while also not decreasing QoE due to packet loss. In this study, content tampering against AES-128 was investigated in a simulated environment. Additionally, the difference in data loss due to packet loss while AES-128 is applied is investigated. The findings highlight that AES-128 segment encryption does not have a noticeable impact on data delivery in low to moderate packet loss conditions while effectively providing confidentiality and practical resistance to content tampering by preventing attackers from producing meaningful manipulated video segments. However, it does not offer formal cryptographic integrity or authenticity guarantees. The results further indicate that while AES-128 segment encryption mitigates content-level tampering, playlist-level manipulation remains a viable attack vector without additional authentication mechanisms.

This study can be extended further by investigating the integration of AES-192 and AES-256 encryptions into HLS, given that these encryption standards are not natively supported.

Author Contributions: The authors confirm that the following individuals contributed to the paper: Study conception and design, B.S.S.; investigation, B.S.S.; analysis and interpretation of results, B.S.S.; writing, B.S.S.; supervision, A.A.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: All data generated in this study are included in the Appendix A of this paper.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AES	Advanced Encryption Standard
DLSS	Deep Learning-based Soft Sensor
CBC	Cipher Block Chaining
CDN	Content Distribution Network
DASH	Dynamic Adaptive Streaming over HTTP
DES	Data Encryption Standard
GNS3	Graphical Network Simulator 3
HAS	HTTP Adaptive Streaming
HDS	HTTP Dynamic Streaming
HLS	HTTP Live Streaming
HTTP	Hypertext Transfer Protocol
ISP	Internet Service Provider
IV	Initialization Vectors
LVS	Live Video Streaming
MitM	Man-in-the-Middle
MSS	Microsoft Smooth Streaming
PSNR	Peak Signal-to-Noise Ratio
QoE	Quality of Experience
QoS	Quality of Service
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
VoD	Video on Demand

Appendix A

Table A1. Data loss in numerical values from Scenario 2.

PLR %	Playlist Length of 10 Segments		Playlist Length of 5 Segments	
	First Run	Second Run	First Run	Second Run
1%	0.19%	0.10%	0.02%	0.03%
2%	0.03%	0.07%	0.04%	0.07%
3%	0.08%	0.07%	0.08%	0.19%
4%	0.15%	0.17%	0.61%	0.52%
5%	7.84%	7.11%	7.83%	8.49%
6%	20.45%	19.46%	23.51%	21.21%

References

1. Cisco Visual Networking Index. Cisco Visual Networking Index: Forecast and Trends, 2017–2022. *White Paper*, 2018. Available online: https://www.futuretimeline.net/data-trends/pdfs/cisco-2017-2022.pdf (accessed on 25 August 2025).
2. Dao, N.-N.; Tran, A.-T.; Tu, N.H.; Thanh, T.T.; Bao, V.N.Q.; Cho, S. A Contemporary Survey on Live Video Streaming from a Computation-Driven Perspective. *ACM Comput. Surv.* **2022**, *54*, 1–38. [CrossRef]
3. Boeckx, B. Optimizing HTTP Adaptive Streaming Using Modern Protocols. 2024. Available online: http://hdl.handle.net/1942/44159 (accessed on 25 August 2025).
4. Andress, J. *Foundations of Information Security: A Straightforward Introduction*; No Starch Press: San Francisco, CA, USA, 2019; ISBN 1718500041.
5. Al-Jabali, M.F. *Image Encryption System by Generating Chains from the Secret Key*; Middle East University: Amman, Jordan, 2016.
6. Pfleeger, C.P. *Security in Computing*; Pearson Education: Delhi, India, 2009; ISBN 8131727254.

7. Punchihewa, A.; Bailey, D. A Review of Emerging Video Codecs: Challenges and Opportunities. In *Proceedings of the 2020 35th International Conference on Image and Vision Computing New Zealand (IVCNZ), Wellington, New Zealand, 25–27 November 2020*; IEEE: New York, NY, USA, 2020; pp. 1–6.
8. Stallings, W. *Cryptography and Network Security: Principles and Practice*; Pearson: Upper Saddle River, NJ, USA, 2020; ISBN 9780136707226.
9. Akhtar, N.; Saddique, M.; Asghar, K.; Bajwa, U.I.; Hussain, M.; Habib, Z. Digital Video Tampering Detection and Localization: Review, Representations, Challenges and Algorithm. *Mathematics* **2022**, *10*, 168. [CrossRef]
10. Bouraqia, K.; Sabir, E.; Sadik, M.; Ladid, L. Quality of Experience for Streaming Services: Measurements, Challenges and Insights. *IEEE Access* **2020**, *8*, 13341–13361. [CrossRef]
11. Bentaleb, A.; Taani, B.; Begen, A.C.; Timmerer, C.; Zimmermann, R. A Survey on Bitrate Adaptation Schemes for Streaming Media over HTTP. *IEEE Commun. Surv. Tutor.* **2019**, *21*, 562–585. [CrossRef]
12. Kesavan, S.; Saravana Kumar, E.; Kumar, A.; Vengatesan, K. An Investigation on Adaptive HTTP Media Streaming Quality-of-Experience (QoE) and Agility Using Cloud Media Services. *Int. J. Comput. Appl.* **2021**, *43*, 431–444. [CrossRef]
13. Menon, V.V.; Premkumar, A.; Rajendran, P.T.; Wieckowski, A.; Bross, B.; Timmerer, C.; Marpe, D. Energy-Efficient Adaptive Video Streaming with Latency-Aware Dynamic Resolution Encoding. In Proceedings of the 3rd Mile-High Video Conference, Denver, CO, USA, 11–14 February 2024; pp. 21–27.
14. Khan, K. Decentralized Identity and User Privacy in Adaptive Video Streaming: Navigating the Blockchain Frontier. *Int. J. Multidiscip. Res. Publ. (IJMRAP)* **2024**, *6*, 190–196.
15. Kalan, R.; Dulger, I. A Survey on QoE Management Schemes for HTTP Adaptive Video Streaming: Challenges, Solutions, and Opportunities. *IEEE Access* **2024**, *12*, 170803–170839. [CrossRef]
16. Oyman, O.; Singh, S. Quality of Experience for HTTP Adaptive Streaming Services. *IEEE Commun. Mag.* **2012**, *50*, 20–27. [CrossRef]
17. Pantos, R.; May, W. HTTP Live Streaming. Available online: https://www.rfc-editor.org/rfc/rfc8216 (accessed on 1 March 2025).
18. Lyko, T.; Broadbent, M.; Race, N.; Nilsson, M.; Farrow, P.; Appleby, S. Improving Quality of Experience in Adaptive Low Latency Live Streaming. *Multimed. Tools Appl.* **2024**, *83*, 15957–15983. [CrossRef]
19. Sabir, B.S.; Mohammad, A.A. Improving Live Streaming QoE Through HLS Parameter Tuning and Load Balancing to Mitigate Packet Loss. *Kurd. J. Appl. Res.* **2025**, *10*, 77–92. [CrossRef]
20. Yun, J.; Kim, M. JLVEA: Lightweight Real-Time Video Stream Encryption Algorithm for Internet of Things. *Sensors* **2020**, *20*, 3627. [CrossRef] [PubMed]
21. Alawi, A.R.; Hassan, N.F. A Proposal Video Encryption Using Light Stream Algorithm. *Eng. Technol. J.* **2021**, *39*, 184–196. [CrossRef]
22. Liu, Y.; Wang, L.; Qouneh, A.; Fu, X. Enabling PIM-Based AES Encryption for Online Video Streaming. *J. Syst. Archit.* **2022**, *132*, 102734. [CrossRef]
23. Guo, R.; Liu, H.; Liu, D. When Deep Learning-Based Soft Sensors Encounter Reliability Challenges: A Practical Knowledge-Guided Adversarial Attack and Its Defense. *IEEE Trans. Ind. Inform.* **2023**, *20*, 2702–2714. [CrossRef]
24. Usmani, M.W.; Shannigrahi, S.; Zink, M. Secure the Stream, Not the Hosts: Attribute-Based Encryption for DRM Enabled Video Streaming. In Proceedings of the 16th ACM Multimedia Systems Conference, Stellenbosch, South Africa, 31 March–4 April 2025; pp. 190–200.
25. Gomez, J.; Kfoury, E.F.; Crichigno, J.; Srivastava, G. A Survey on Network Simulators, Emulators, and Testbeds Used for Research and Education. *Comput. Netw.* **2023**, *237*, 110054. [CrossRef]
26. EnvoyProxy Supported Load Balancers. Available online: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers (accessed on 1 March 2025).
27. Amirpour, H.; Zhu, J.; Le Callet, P.; Timmerer, C. A Real-Time Video Quality Metric for HTTP Adaptive Streaming. In *Proceedings of the ICASSP 2024—2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, Republic of Korea, 14–19 April 2024*; IEEE: New York, NY, USA, 2024; pp. 3810–3814.
28. Setiadi, D.R.I.M. PSNR vs SSIM: Imperceptibility Quality Assessment for Image Steganography. *Multimed. Tools Appl.* **2021**, *80*, 8423–8444. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.